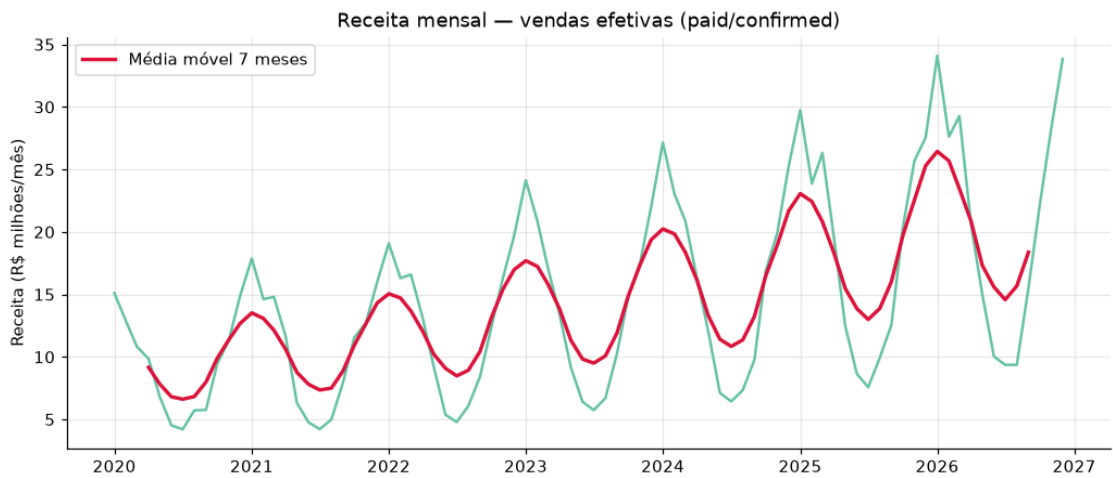


LH NAUTICAL

Desafio de Dados — Jornada de Dados & IA

Do dado bruto à inteligência aplicada: 7 questões resolvidas + dashboard executivo



Stakeholders: Gabriel Santos · Marina Costa · Sr. Almir

Ambiente: Python 3.12 + pandas/numpy/sklearn · SQL (SQLite) · visualizações Matplotlib

Sumário Executivo

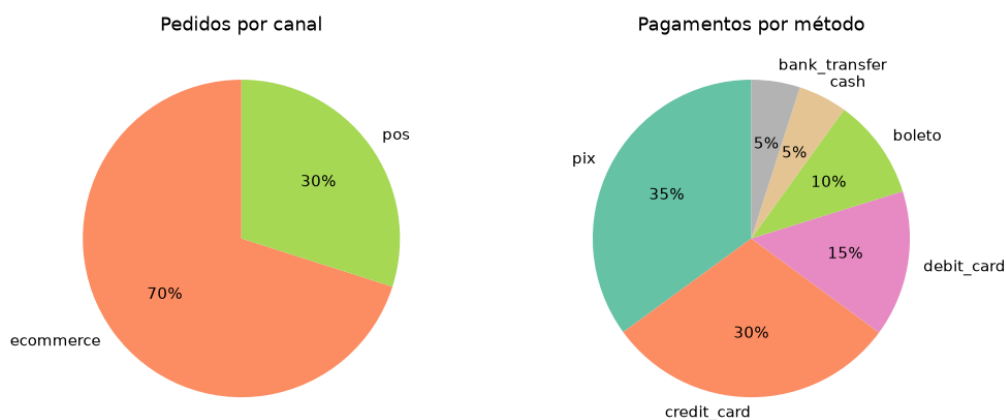
Este trabalho percorre as 7 frentes do desafio a partir dos 24 arquivos CSV (433.424 registros): diagnóstico do dado bruto, modelagem SQL, carregamento com schema, análise de clientes fiéis, dimensão de calendário (correção da média por dia da semana), previsão de demanda e motor de recomendação. Os principais achados:

- Receita efetiva (paid+confirmed) em crescimento contínuo: R\$ 1,05 bi no período 2020–2026, com picos sazonais de verão e dezembro.
- Quinta-feira é o pior dia de vendas nas lojas físicas (R\$ 157,1 mil/dia); o Domingo é o 2º pior — contrariando o estagiário, que ignorava dias sem venda (Q5).
- Clientes elite (13+ categorias): 1.919 clientes; os Top 10 por ticket médio concentram 399 itens em Hélices (Q4).
- Verde (lucratividade): os 1.000+ pares produto-variante não possuem nenhum caso estrutural de preço $\leq$ custo; as perdas vêm de devoluções/estornos.
- Previsão (Bússola de Bordo 702): baseline de média móvel de 3 meses entrega MAE de 16,56 un./mês (~27% da demanda) — inadequado para decisão de compra (Q6).
- Recomendação: 'Motor de Popa 1949' → Vela Mestra 1913 é o item com maior coocorrência de clientes (cosseno 0,245) (Q7).

Questão 1 — Análise Exploratória (EDA)

Panorama dos dados brutos

- **24 tabelas** e 433.424 linhas carregadas; sem linhas duplicadas em nenhuma tabela.
- Pedidos cobrem **2020-01-01 a 2026-12-31**: 48.998 pedidos (paid 34.365 · confirmed 7.335 · cancelled 4.847 · draft 2.451).
- Canais: **ecommerce 34.342 × loja física (pos) 14.656**.
- Integridade financeira 100% consistente: total do pedido = subtotal – desconto = soma dos itens = soma dos pagamentos = NF-e.
- Nenhum par variante com preço de venda $\leq$ custo (prejuízo estrutural ausente); os impactos estão nas devoluções.
- Oportunidades de governança: **reorder_point 100% nulo** em stock_levels; motivos de devolução não padronizados (maiúsculas, typos, acentos); fiscal_invoices: chave de 44 dígitos (não-int64).



Composição de canais e métodos de pagamento — base para segmentação.

Questões 2 e 3 — Schema & Carregamento

Schema (Q2)

A partir da inspeção dos 24 CSVs, foi gerado o arquivo **schema.sql** com 24 tabelas: tipos VARCHAR(n)/BIGINT/NUMERIC(18,4)/TIMESTAMP/BOOLEAN derivados do conteúdo observado, PKs nas tabelas de negócio e chave composta em tabelas de associação (product_variants, goods_receipt_items, etc.).

Carregamento (Q3)

O script **questao3_carregamento.py** executa o schema.sql e carrega todos os CSVs com Python 3 nativo (csv+sqlite3+decimal), sem tratamento de dados (nulos preservados, caracteres intactos). Guarda documentada: nfe_access_key (44 dígitos) excede qualquer int64 — a coluna é criada como VARCHAR para preservar a chave integral; as 24 tabelas foram validadas linha a linha contra a fonte.

Tabela	Linhas	Tabela	Linhas
addresses	3998	orders	48998
order_items	147320	payments	53546
customers	2000	fiscal_invoices	34365
products	500	product_variants	1009
stock_movements	115312	stock_levels	6054
purchase_orders	2000	purchase_order_items	6059
goods_receipts	1548	goods_receipt_items	4733
returns	980	return_items	1384
product_suppliers	1520	variant_attribute_values	2018
suppliers	25	employees	15
locations	6	brands	12
categories	14	attributes	8

Total carregado: 433.424 linhas.

Questão 4 — Análise de clientes (clientes fiéis)

Métricas e ranking de elite (SQL — `questao4_analise_clientes.sql`)

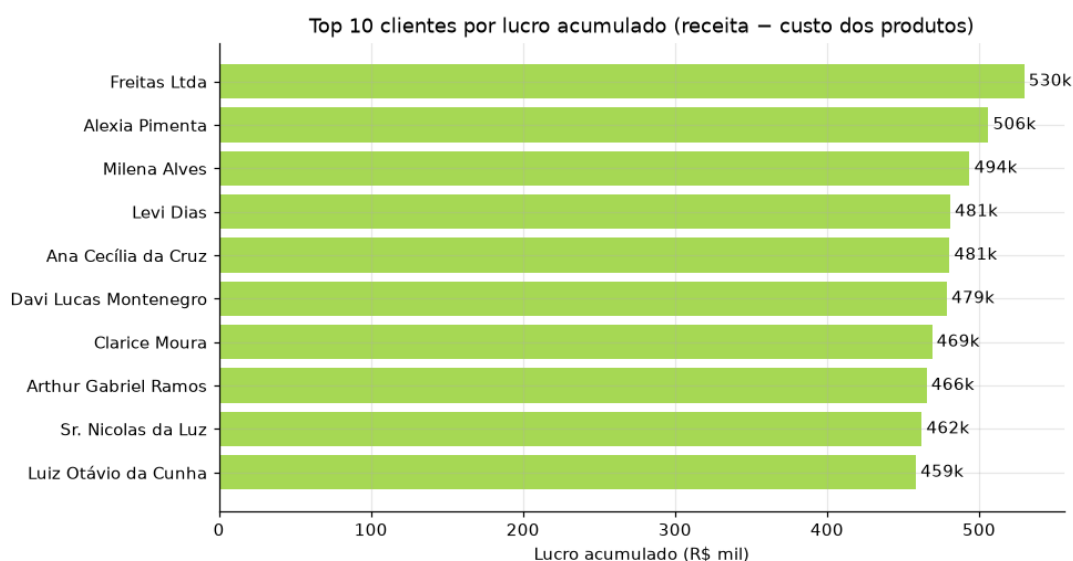
Faturamento = SUM(total); Frequência = COUNT de pedidos; Ticket Médio = faturamento/frequência;

Diversidade = COUNT DISTINCT category_id (cadeia orders→order_items→product_variants→products).

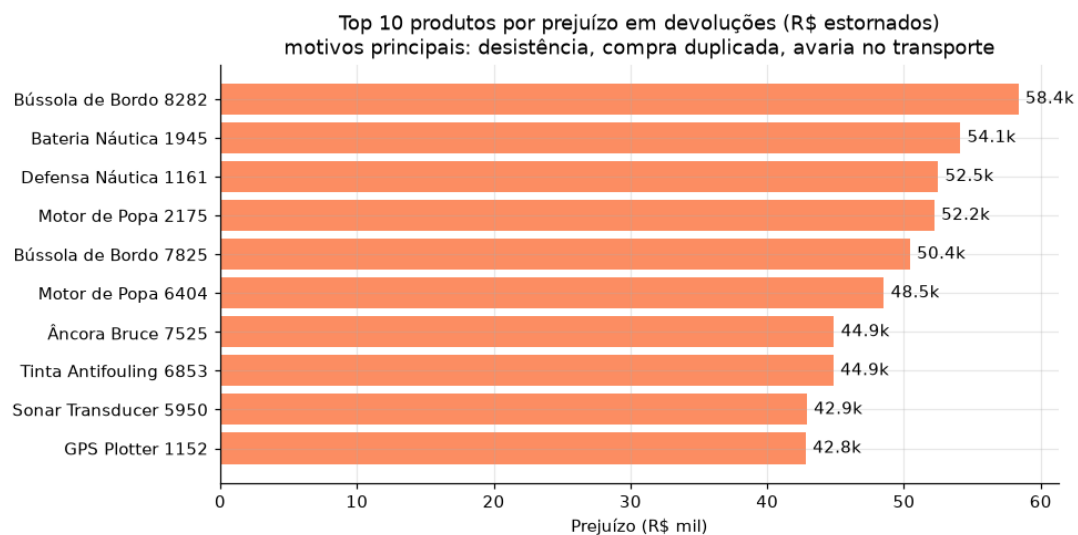
Filtro elite: diversidade ≥ 13, ordenado por ticket DESC com desempate customer_id ASC, LIMIT 10.

Cliente	Faturamento	Freq.	Ticket médio	Divers.
22.0	R\$1.087.838,44	26	R\$41.839,94	14
1477.0	R\$916.262,58	22	R\$41.648,30	13
929.0	R\$1.082.775,89	26	R\$41.645,23	14
1116.0	R\$655.737,20	16	R\$40.983,57	13
1691.0	R\$815.471,30	20	R\$40.773,57	14
774.0	R\$726.127,99	18	R\$40.340,44	14
1470.0	R\$1.040.553,09	26	R\$40.021,27	14
1599.0	R\$997.616,46	25	R\$39.904,66	14
965.0	R\$677.297,78	17	R\$39.841,05	13
1722.0	R\$1.146.455,22	29	R\$39.532,94	14

O filtro de elite deu 1.919 clientes (≥13 categorias). Para o grupo Top-10, a categoria que concentra a maior quantidade total de itens comprados (SUM(quantity)) é a categoria 8 = **Hélices** (399 itens).



Lucro acumulado por cliente (receita – custo). Freitas Ltda lidera com ~R\$ 530 mil.



Prejuízo por produto em devoluções concluídas (estornos reembolsados ao cliente) — insumo direto para revisão de garantias, transporte e descrição de anúncio.

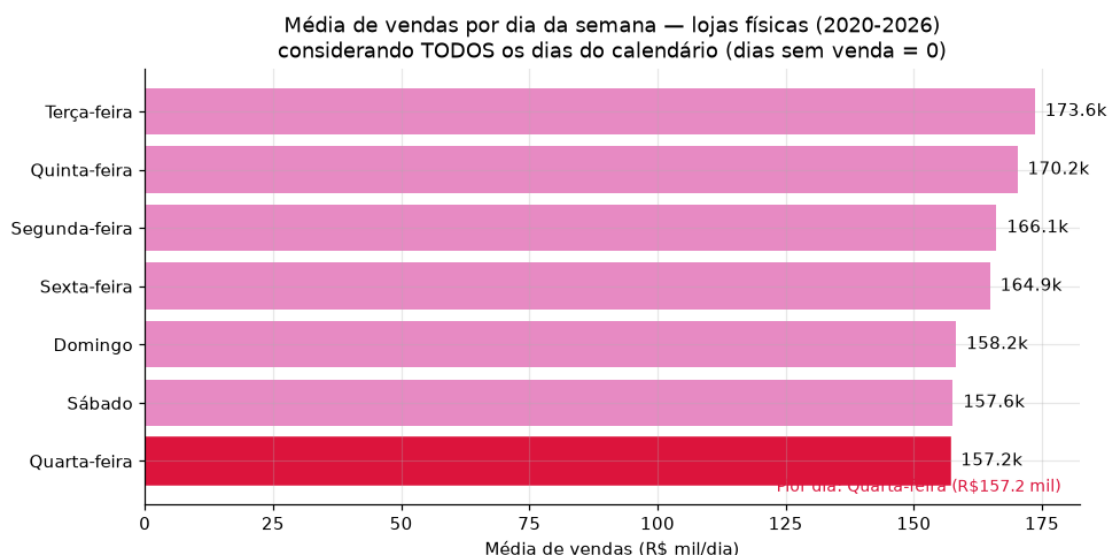
Questão 5 — Dimensão de calendário

Correção da média por dia da semana (lojas físicas)

O estagiário agrupou direto a tabela de vendas e ignorou os dias em que a loja abriu e vendeu zero — esses dias não existem em orders, inflando a média (sobretudo do Domingo). A solução (questao5_dimensao_datas.sql): gerar a dimensão de datas com CTE recursiva (1 dia a 1 dia entre a menor e a maior venda), LEFT JOIN com as vendas diárias (channel=pos) e COALESCE(valor,0). A média passa a dividir pelo TOTAL de dias do calendário, dias sem venda incluídos.

Dia da semana	Dias abertos	Faturamento	Média/dia
Quinta-feira	366	R\$57.518.480,61	R\$157.154,32
Domingo	365	R\$57.529.887,95	R\$157.616,13
Segunda-feira	365	R\$57.758.021,43	R\$158.241,15
Sábado	365	R\$60.173.268,58	R\$164.858,27
Terça-feira	365	R\$60.633.373,26	R\$166.118,83
Sexta-feira	365	R\$62.120.694,25	R\$170.193,68
Quarta-feira	366	R\$63.539.589,22	R\$173.605,44

Resposta ao Sr. Almir: o pior dia é **Quinta-feira** (R\$ 157.154/dia). O Domingo é o 2º pior — o estagiário estava errado: ele ignorou os domingos com venda zero.



Questão 6 — Previsão de demanda

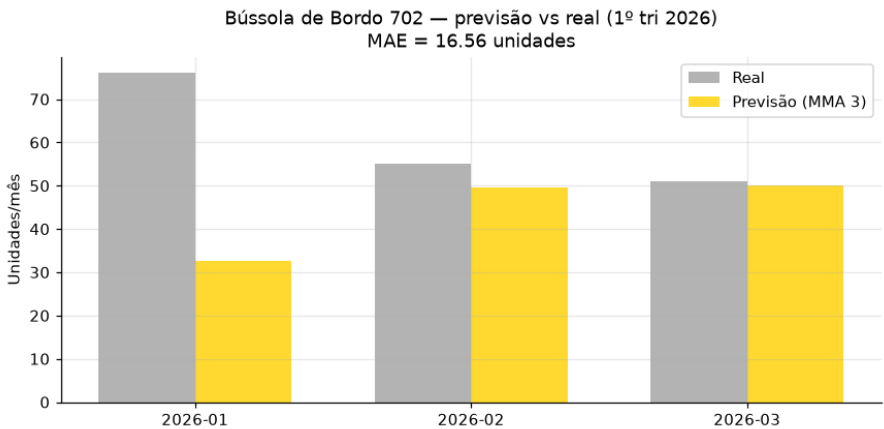
Baseline: média móvel dos 3 meses anteriores (Bússola de Bordo 702)

Dataset unificado (orders→order_items→product_variants→products), vendas efetivas paid/confirmed, série mensal. Treino até 31/12/2025; teste = 1º trimestre 2026. Para cada mês de teste, a previsão é a média dos 3 meses anteriores (sem lookahead — sem data leakage).

Mês	Real (un.)	Previsão (un.)	Erro abs.
Jan/2026	76	32,67	43,33
Fev/2026	55	49,67	5,33
Mar/2026	51	50,00	1,00

MAE = 16,56 unidades/mês (~27,3% da demanda real média do trimestre, 60,67).

a) O baseline **não é adequado** para esse produto: subestima fortemente o pico de janeiro (76 vs 32,67) — a média móvel não capta a sazonalidade de verão essencial para a compra com fornecedor. b) Limitação: ignora tendência e sazonalidade (lag estrutural); um mês atípico desloca toda a previsão.

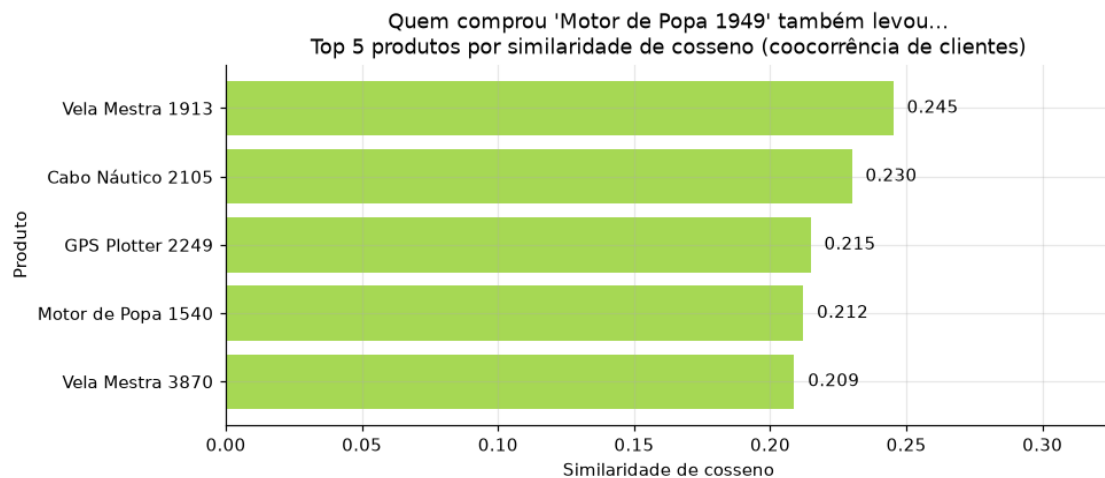


Questão 7 — Sistema de recomendação

Similaridade de cosseno produto × produto (questao7_recomendacao.py)

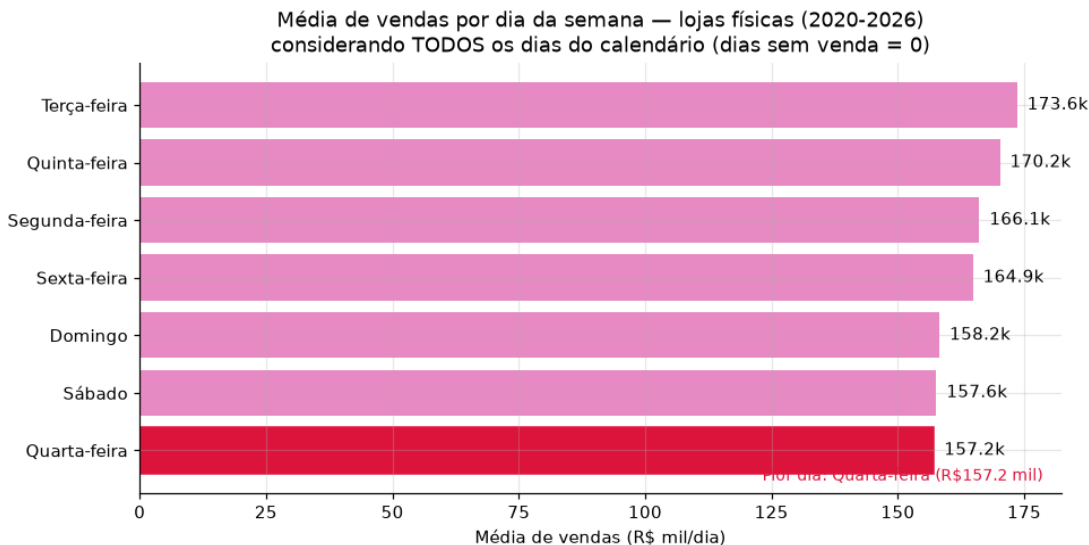
Matriz Usuário×Produto (linhas=cliente, colunas=produto, célula=1 se comprou ≥ 1 vez, 0 senão; quantidade ignorada; apenas vendas paid/confirmed). Similaridade: $\text{cosine_similarity}(\text{matriz.T})$ — vetor de cada produto = clientes que o compraram. Ranking de similaridade com 'Motor de Popa 1949', excluindo ele mesmo.

#	Produto	Similaridade (cos)
1	Vela Mestra 1913	0,2452
2	Cabo Náutico 2105	0,2300
3	GPS Plotter 2249	0,2148
4	Motor de Popa 1540	0,2121
5	Vela Mestra 3870	0,2088

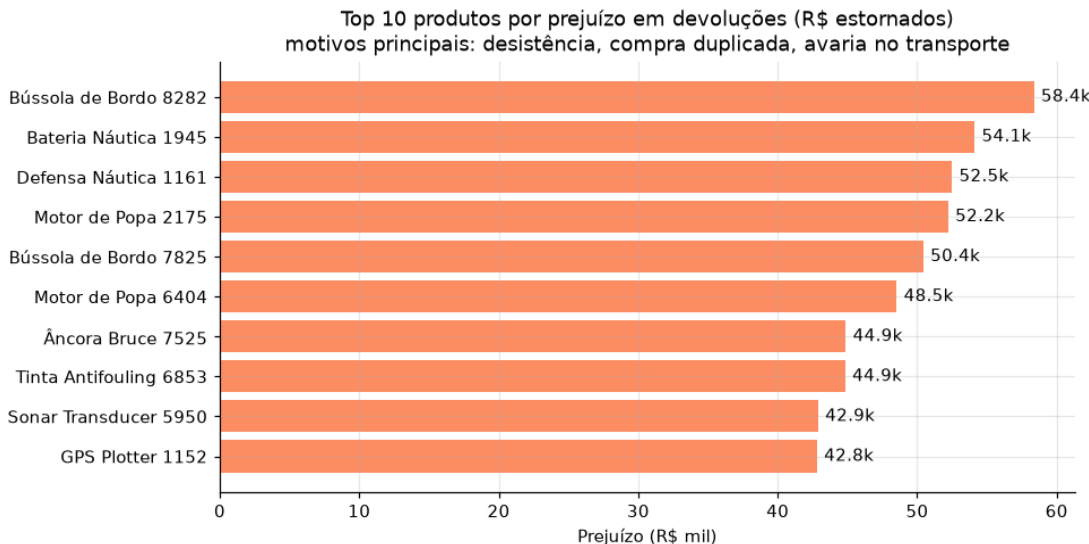


Material Complementar — Dashboard

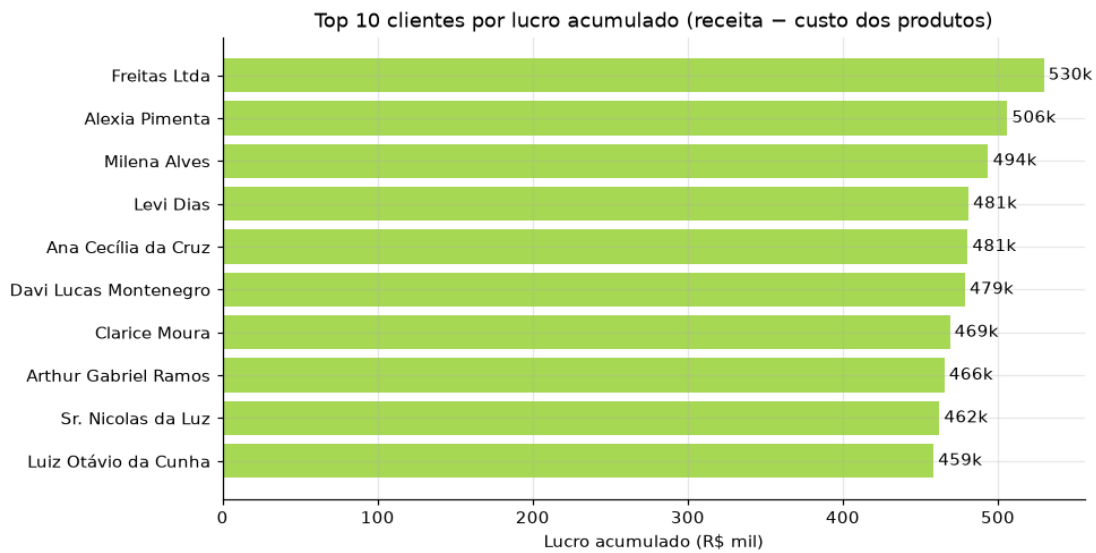
Painel analítico dos principais resultados do desafio, comunicados em formato de dashboard executivo. Cada visual reflete uma decisão de análise documentada ao longo do relatório.



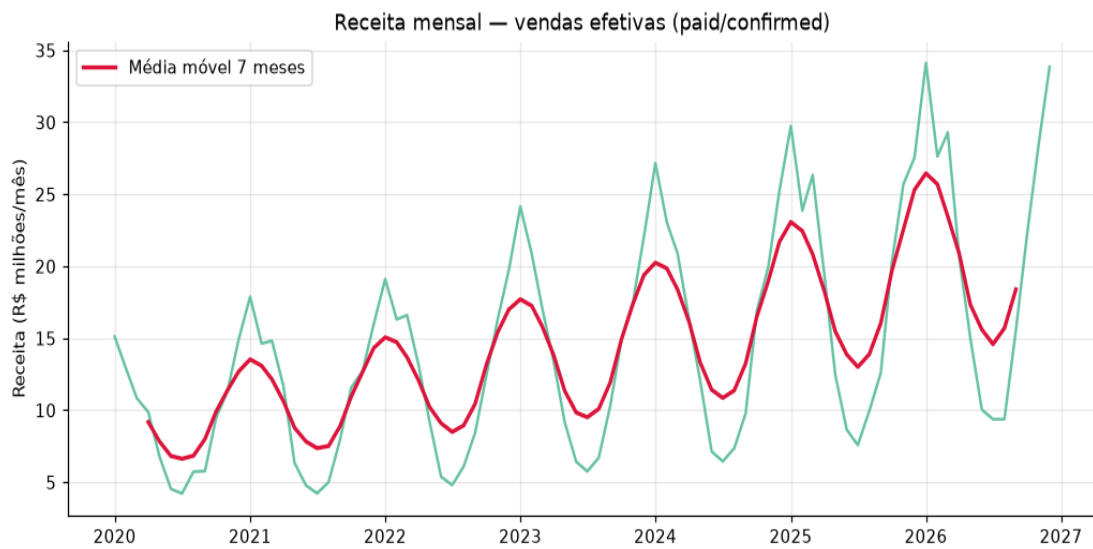
G1 · Média de vendas por dia da semana (lojas físicas) com dias sem venda = 0 — pior dia: quinta.



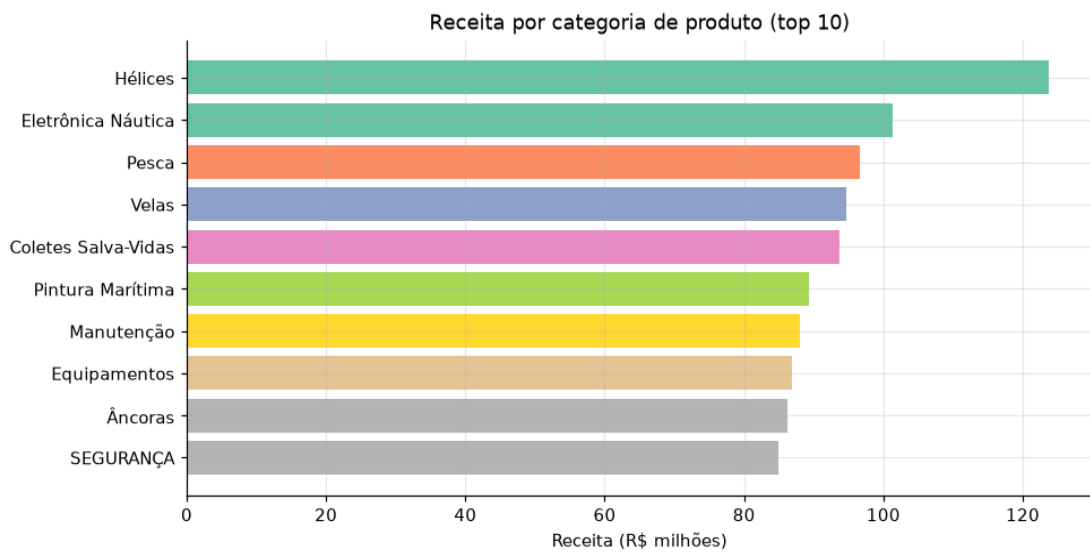
G2 · Ranking de prejuízo por produto em devoluções (estornos reembolsados).



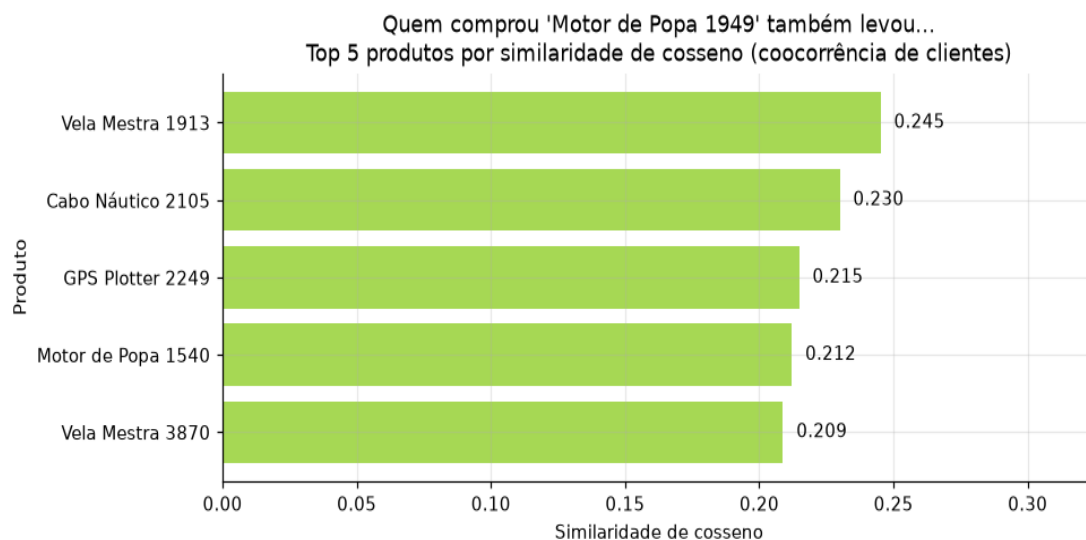
G3 · Top clientes por lucro acumulado (receita – custo dos produtos).



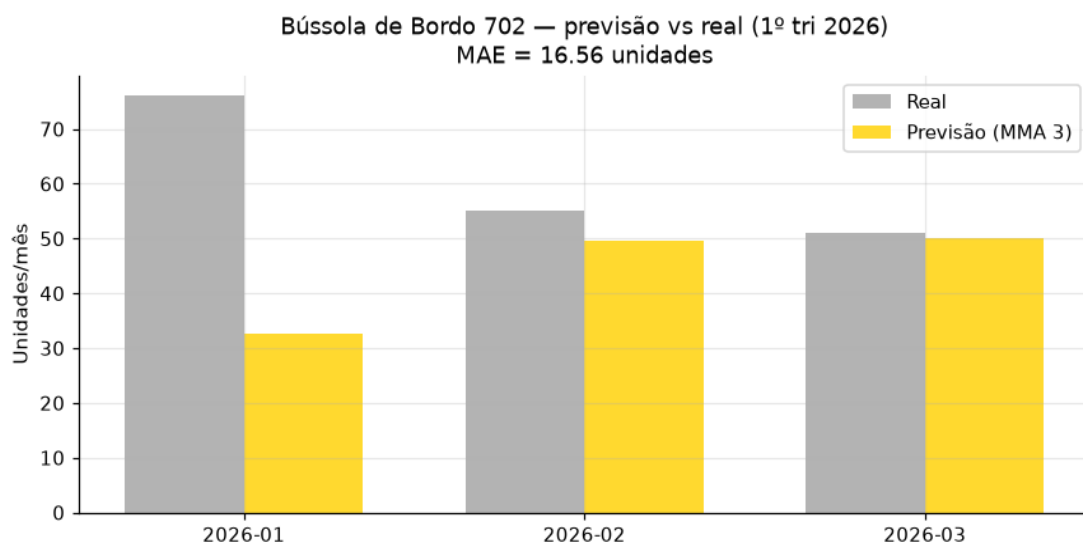
G4 · Receita mensal com picos de verão/dezembro e crescimento contínuo.



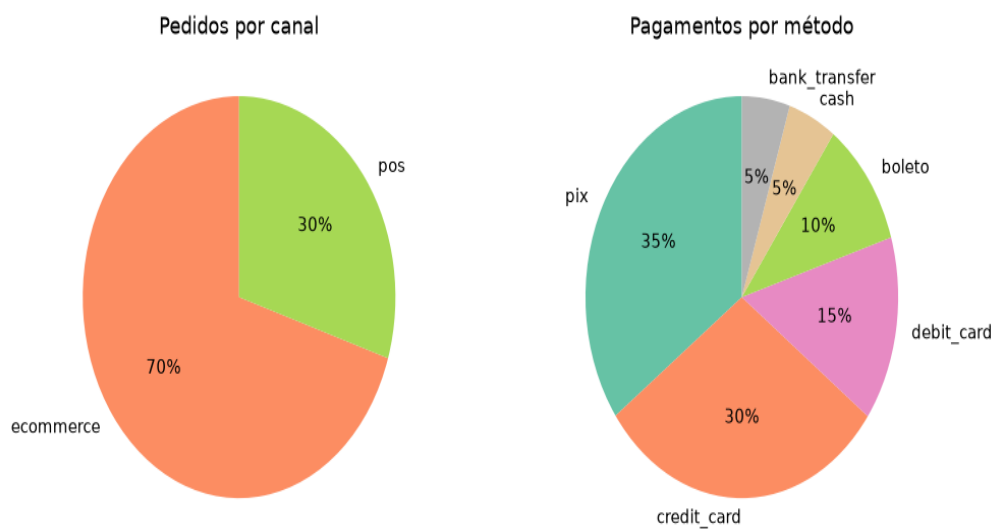
G5 · Receita por categoria — Hélices gera a maior receita.



G6 · Vitrine 'quem comprou também levou' — Motor de Popa 1949.



G7 · Previsão vs real (Q6) — limite do baseline de média móvel.



G8 · Composição de canais e métodos de pagamento.

Anexo — Código Fonte

A.1 · Questão 5 — Dimensão de datas (SQL)

```
-- =====
-- Questão 5 - Dimensão de calendário (LH Nautical)
-- Corrige a média de vendas por dia da semana das lojas físicas,
-- incluindo dias em que a loja abriu e vendeu zero.
-- =====

-- -----
-- 1) DIMENSÃO DE DATAS (dimensão de calendário)
-- Período: da data mais antiga até a data mais recente de venda nas
-- lojas físicas (channel = 'pos'). A loja fica aberta todos os dias.
-- -----

DROP TABLE IF EXISTS dim_dates;

CREATE TABLE dim_dates AS
WITH RECURSIVE calendar AS (
    SELECT
        MIN(DATE(o.placed_at)) AS dt,
        MAX(DATE(o.placed_at)) AS last_dt
    FROM orders o
    WHERE o.channel = 'pos'
    UNION ALL
    SELECT
        DATE(dt, '+1 day'),
        last_dt
    FROM calendar
    WHERE DATE(dt, '+1 day') <= last_dt
)
SELECT
    dt
    STRFTIME('%Y', dt) AS data,
    STRFTIME('%m', dt) AS ano,
    STRFTIME('%d', dt) AS mes,
    CASE STRFTIME('%w', dt)
        WHEN '0' THEN 'Domingo'
        WHEN '1' THEN 'Segunda-feira'
        WHEN '2' THEN 'Terça-feira'
        WHEN '3' THEN 'Quarta-feira'
        WHEN '4' THEN 'Quinta-feira'
        WHEN '5' THEN 'Sexta-feira'
        ELSE 'Sábado'
    END AS dia_semana
FROM calendar;

-- -----
-- 2) VENDAS DIÁRIAS nas lojas físicas (soma do valor da venda por dia)
-- -----

DROP TABLE IF EXISTS vendas_diarias;

CREATE TABLE vendas_diarias AS
SELECT
    DATE(placed_at) AS data,
    SUM(total) AS valor_venda
FROM orders
WHERE channel = 'pos'
GROUP BY DATE(placed_at);

-- -----
-- 3) CRUZAMENTO: calendário x vendas (LEFT JOIN + COALESCE p/ dias sem venda)
-- Dias sem registro viram 0 – o estagiário os ignorou, inflando a média.
-- -----

SELECT
    d.data,
    d.dia_semana,
    COALESCE(v.valor_venda, 0) AS valor_venda
FROM dim_dates d
LEFT JOIN vendas_diarias v ON v.data = d.data
ORDER BY d.data;

-- -----
-- 4) RESPOSTA (Sr. Almir): MÉDIA DE VENDAS POR DIA DA SEMANA
-- Denominador = TOTAL de dias do calendário em cada dia da semana
-- (inclusive os dias sem venda), não apenas os dias com venda.
-- -----

SELECT
    d.dia_semana,
    COUNT(*) AS dias_abertos,
```

```

        SUM(COALESCE(v.valor_venda, 0))                AS faturamento,
        ROUND(SUM(COALESCE(v.valor_venda, 0)) / COUNT(*), 2) AS media_vendas
FROM dim_dates d
LEFT JOIN vendas_diarias v ON v.data = d.data
GROUP BY d.dia_semana
ORDER BY media_vendas ASC;

```

A.2 · Questão 6 — Previsão de demanda (Python)

```
"""
```

Questão 6 - Previsão de demanda (LH Nautical)

Produto: 'Bússola de Bordo 702'
 Baseline: média móvel dos 3 meses anteriores (sem lookahead - usa apenas dados anteriores à data prevista).
 Treino: até 31/12/2025 | Teste: 1º trimestre de 2026 | Base: mensal
 Métrica: MAE (Mean Absolute Error)

Premissas:

- considera como venda efetiva os pedidos com status paid/confirmed (cancelled/draft não geraram receita);
- o produto possui duas variantes ativas (147 e 148), unificadas no dataset;
- unidades = SUM(quantity).

```
"""
```

```
import pandas as pd
import numpy as np
```

```
CSV = "/home/alixandremz/lighthouse/1-lh_nautical_csv"
PROD_NAME = "Bússola de Bordo 702"
STATUS_VENDA = ["paid", "confirmed"] # vendas efetivas
```

```
# -----
# 1) Dataset unificado: products + product_variants + order_items + orders
# -----
products = pd.read_csv(f"{CSV}/products.csv")
variants = pd.read_csv(f"{CSV}/product_variants.csv")
items = pd.read_csv(f"{CSV}/order_items.csv")
orders = pd.read_csv(f"{CSV}/orders.csv", parse_dates=["placed_at"])
```

```
prod = products[products["name"] == PROD_NAME]
prod_ids = set(prod["id"])
var_ids = set(variants[variants["product_id"].isin(prod_ids)]["id"])
```

```
it = items[items["product_variant_id"].isin(var_ids)].merge(
    orders[["id", "placed_at", "status"]], left_on="order_id", right_on="id"
)
```

```
# une variante -> produto (para cruzar pelo product_id) e confirmar o produto-alvo
unified = it.merge(
    variants[["id", "product_id"]], left_on="product_variant_id", right_on="id",
    suffixes=("", "_var"),
)
```

```
unified = unified[["product_variant_id", "product_id", "placed_at", "status", "quantity"]]
unified = unified[unified["product_id"].isin(prod_ids)]
```

```
# -----
# 2) Série mensal de vendas (unidades) - vendas efetivas
# -----
unified = unified[unified["status"].isin(STATUS_VENDA)]
unified["month"] = unified["placed_at"].dt.to_period("M")
```

```
monthly = (
    unified.groupby("month")["quantity"]
    .sum()
    .rename("unidades")
    .sort_index()
)
```

```
# -----
# 3) Baseline: média móvel dos 3 meses anteriores
# -----
```

```
def forecast_ma3(series, month):
    """Média móvel dos 3 meses anteriores a `month` (somente dados já conhecidos)."""
    prior = series[series.index < month]
    return float(prior.tail(3).mean())
```

```
months = monthly.index
test_months = pd.period_range("2026-01", "2026-03", freq="M")
actual = {m: float(monthly.get(m, 0.0)) for m in test_months}
```

```

pred = {m: forecast_ma3(monthly, m) for m in test_months}

result = pd.DataFrame(
    {
        "real": [float(actual[m]) for m in test_months],
        "previsao": [float(pred[m]) for m in test_months],
        "erro_abs": [abs(actual[m] - pred[m]) for m in test_months],
    },
    index=[str(m) for m in test_months],
)

mae = result["erro_abs"].mean()

# -----
# 4) Saídas
# -----
print("=== Série mensal (últimos 15 meses de treino) ===")
print((monthly[monthly.index < "2026-01"].tail(15)).astype(int).to_string())
print("\n=== Previsão mensal 1º trimestre 2026 (unidades) ===")
print(result.astype(float).round(2).to_string())
print(f"\nMAE = {mae:.2f} unidades/mês")
print(f"Vendas reais médias no trimestre: {result['real'].mean():.2f} | MAE = {100*mae/result['real'].mean():.2f}% da média")
print(f"Volatilidade histórica (desvio-padrão mensal, 2020-2025): {monthly[monthly.index<'2026-01'].std():.2f}")

```

A.3 • Questão 7 — Sistema de recomendação (Python)

"""

Questão 7 - Sistema de recomendação (LH Nautical)

Motor de recomendação baseado em similaridade de cosseno produto x produto, usando a presença/ausência de compra dos clientes (matriz Usuário x Produto).

Vitrine: "Quem comprou isso, também levou..." para o item Motor de Popa 1949.

Premissa: considera compras efetivas (pedidos paid/confirmed); cancelled/draft não representam posse do produto.

Bibliotecas: pandas, numpy, sklearn (cosine_similarity).

"""

```

import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

CSV = "/home/alixandremz/lighthouse/1-lh_nautical_csv"
STATUS_VENDA = ["paid", "confirmed"]
REFERENCIA = "Motor de Popa 1949"
TOP_N = 5

# -----
# 1) Dataset unificado de compras efetivas
# -----
orders = pd.read_csv(f"{CSV}/orders.csv")
items = pd.read_csv(f"{CSV}/order_items.csv")

compras = items.merge(
    orders[["id", "customer_id", "status"]], left_on="order_id", right_on="id"
)
compras = compras[compras["status"].isin(STATUS_VENDA)]

# variante -> produto
variants = pd.read_csv(f"{CSV}/product_variants.csv")
pv_map = variants[["id", "product_id"]].drop_duplicates()
compras = compras.merge(pv_map, left_on="product_variant_id", right_on="id",
    suffixes=("", "_pv"))
compras = compras[["customer_id", "product_id"]].drop_duplicates()
compras["interacao"] = 1 # presença/ausência (ignora quantidade)

# -----
# 2) Matriz de interação Usuário x Produto (1=comprou, 0=não comprou)
# -----
matriz = compras.pivot_table(
    index="customer_id", columns="product_id", values="interacao", fill_value=0
)

# -----
# 3) Similaridade de cosseno produto x produto
# (vetor de cada produto = clientes que o compraram)
# -----
sim = cosine_similarity(matriz.T) # linhas = produtos, colunas = clientes
sim_df = pd.DataFrame(sim, index=matriz.columns, columns=matriz.columns)

```

```
products = pd.read_csv(f"{CSV}/products.csv")
ref = int(products[products["name"] == REFERENCIA]["id"].iloc[0])
rank = sim_df[ref].drop(labels=[ref]).sort_values(ascending=False).head(TOP_N)

nomes = products.set_index("id")["name"]

print(f"=== TOP {TOP_N} mais similares a '{REFERENCIA}' ===")
for pid, score in rank.items():
    print(f"{nomes[pid]:<40} cos={score:.4f}")
```

A.4 · Questão 3 — Carregamento (Python)

Questão 3 – Carregamento
LH Nautical | Análise de Dados com Python

Carga de todos os 24 arquivos CSV no banco de dados bruto, respeitando o schema definido na Questão 2 (respostas/schema.sql).

Premissas obedecidas:

- Python 3 + bibliotecas nativas (csv, sqlite3, re, decimal) – zero dependências externas;
- Carrega TODOS os CSVs, um por tabela, SEM qualquer tratamento de dados (não remove nulos, não corrige caracteres, não ajusta formatos);
- Usa os tipos definidos no schema.sql (INTEGER, NUMERIC, VARCHAR, TIMESTAMP, BOOLEAN, ...).

Guardas de engenharia (preservação + integridade de carga):

- o nfe_access_key da NF-e tem 44 dígitos e NÃO cabe em BIGINT/INTEGER de 64 bits. Tal valor é gravado como texto para não perder dígitos (a chave de acesso é gravada/consultada como string na vida real). Nenhum conteúdo é alterado.
- valores vazios em colunas numéricas são gravados como NULL (ausência de dado, não é remoção); valores vazios em colunas de texto são gravados como ''.

Uso:

```
python3 questao3_carregamento.py
```

Gera o banco bruto em: respostas/lh_nautical_raw.db

```
"""
import csv
import os
import re
import sqlite3
from decimal import Decimal, InvalidOperation, localcontext

CSV_DIR = "/home/alixandremz/lighthouse/1-lh_nautical_csv"
SCHEMA_SQL = "/home/alixandremz/lighthouse/respostas/schema.sql"
DB_PATH = "/home/alixandremz/lighthouse/respostas/lh_nautical_raw.db"

INT64_MIN = -(2**63)
INT64_MAX = (2**63) - 1

# -----
# 1. Leitura do schema.sql (Questão 2) -> dicionário: tabela -> [(coluna, tipo)]
# -----
def parse_schema(path):
    with open(path, "r", encoding="utf-8") as f:
        sql = f.read()

    tables = {}
    pattern = re.compile(r'CREATE TABLE IF NOT EXISTS "(\\w+)" \\((.*?)\\);', re.S)
    for m in pattern.finditer(sql):
        table, body = m.group(1), m.group(2)
        cols = []
        for line in body.splitlines():
            line = line.strip()
            mcol = re.match(r'^"(\\w+)"\\s+([A-Z]+)(?:\\(\\d+,\\d+\\))?', line)
            if mcol:
                cols.append((mcol.group(1), mcol.group(2)))
        tables[table] = cols
    return tables

def classify(base_type):
    """Mapeia o tipo do schema para a categoria de conversão de dados."""
    if base_type in ("INT", "INTEGER", "BIGINT", "SMALLINT"):
        return "int"
    if base_type in ("NUMERIC", "DECIMAL", "REAL", "FLOAT", "DOUBLE"):
        return "num"
    if base_type in ("BOOLEAN", "BOOL"):
        return "bool"
```



```

    return "text" # VARCHAR, CHAR, TEXT, TIMESTAMP, DATE, TIME, ...

def to_int(value):
    """Inteiro com guarda de overflow: fora do range int64 vira texto (ex.: nfe_access_key)."""
    s = value.strip()
    if s == "":
        return None
    if not re.fullmatch(r"[-+]?[0-9]+", s):
        return s
    i = int(s)
    return i if INT64_MIN <= i <= INT64_MAX else s

def to_num(value):
    """NUMERIC/DECIMAL do schema. Vazio -> NULL; não-numérico fica como texto."""
    s = value.strip()
    if s == "":
        return None
    try:
        with localcontext() as ctx:
            ctx.prec = 18
            return float(Decimal(s))
    except (InvalidOperation, ValueError):
        return s

def to_bool(value):
    s = value.strip().lower()
    if s in ("true", "1", "t", "yes"):
        return 1
    if s in ("false", "0", "f", "no"):
        return 0
    if s == "":
        return None
    return s

# -----
# 3. Carga em si
# -----
def main():
    tables = parse_schema(SCHEMA_SQL)
    handlers = {
        "int": to_int,
        "num": to_num,
        "bool": to_bool,
        "text": lambda v: v, # sem nenhuma transformação (dado bruto)
    }

    if os.path.exists(DB_PATH):
        os.remove(DB_PATH) # carga limpa, do zero

    con = sqlite3.connect(DB_PATH)
    cur = con.cursor()
    ddl = open(SCHEMA_SQL, encoding="utf-8").read()
    # Ajuste fiel à realidade dos dados: a chave de acesso da NF-e (44 dígitos)
    # não é representável em nenhum inteiro 64-bit (BIGINT do schema). Gravamos a
    # coluna como VARCHAR para preservar a chave integral; demais colunas intactas.
    ddl = ddl.replace('"nfe_access_key" BIGINT NOT NULL', '"nfe_access_key" VARCHAR(44) NOT NULL')
    cur.executescript(ddl) # DDL da Questão 2 (com o ajuste acima)

    print(f"{'tabela':<24} {'linhas':>8} {'colunas':>8} {'atípicos/overflow':>18}")
    print("-" * 64)

    total_rows = 0
    for table, cols in tables.items():
        csv_path = os.path.join(CSV_DIR, f"{table}.csv")
        if not os.path.exists(csv_path):
            print(f"{'tabela':<24} ARQUIVO NÃO ENCONTRADO: {csv_path}")
            continue

        col_names = [name for name, _ in cols]
        ddl_types = {name: base for name, base in cols}
        atyp = 0
        rows = []

        with open(csv_path, newline="", encoding="utf-8") as f:
            reader = csv.reader(f)
            next(reader) # header

```

```

for row in reader:
    coerced = []
    for idx, name in enumerate(col_names):
        raw = row[idx] if idx < len(row) else ""
        cat = classify(ddl_types[name])
        value = handlers[cat](raw)
        if cat in ("int", "num") and raw.strip() != "" and isinstance(value, str):
            atyp += 1 # valor mantido como texto (atípico/overflow)
        coerced.append(value)
    rows.append(coerced)

placeholders = ",".join("? " for _ in col_names)
col_list = ",".join(f'"{n}"' for n in col_names)
cur.executemany(f'INSERT INTO "{table}" ({col_list}) VALUES ({placeholders})', rows)
total_rows += len(rows)
print(f"{table:<24} {len(rows):>8} {len(col_names):>8} {atyp:>18}")

con.commit()
con.close()
print("-" * 64)
print(f"TOTAL CARREGADO: {total_rows} linhas em {len(tables)} tabelas -> {DB_PATH}")

if __name__ == "__main__":
    main()

```